

Chapter 8 In-Class Exercise: Searching Strings in a Patient Database

Prof. Jordan C. Hanson

September 21, 2026

1 Starting Data

- Chapter 8 string operations allow us to make a database search more flexible. Instead of requiring an exact patient ID, we can search for names and partial names.

Begin with the patient database

```
patients = [  
    [1042, "Maria Garcia", 37],  
    [1057, "Daniel Kim", 52],  
    [1081, "Aisha Patel", 29],  
    [1094, "Elena Garcia", 44],  
    [1103, "Marcus Johnson", 61]  
]
```

Each record contains

```
[patient_id, name, age]
```

2 Exact String Match

- Write a loop that prints the complete patient record whose name is exactly

```
"Maria Garcia"
```

Use the string comparison operator `==`. A useful structure is

```
for patient in patients:  
    if patient[1] == "Maria Garcia":  
        print(patient)
```

How many patient records should be printed?

3 Substring Match

- Modify the search so that every patient whose name contains "Garcia" is printed. Use Python's `in` operator:

```
if "Garcia" in patient[1]:  
    print(patient)
```

- How many records are printed?
- Why does this search find both "Maria Garcia" and "Elena Garcia"?

4 Case-Insensitive Search

- A user may type "garcia", "Garcia", or "GARCIA". Use the string method `.lower()` so that capitalization does not affect the search. Try

```
search = "GARCIA"  
for patient in patients:  
    if search.lower() in patient[1].lower():  
        print(patient)
```

Why must both `search` and the patient name be converted to lowercase?

5 User-Controlled Search

- Ask the user for a complete or partial patient name:

```
search = input("Enter a patient name or partial name: ")
```

Write a loop that searches all patient names and prints every matching record. Your search must be case-insensitive. Test your program using

```
Garcia  
garcia  
Daniel  
PATEL
```

6 Challenge: Count Matches

- Add a counter before the loop:

```
matches = 0
```

Whenever a matching patient is found, increase the counter:

```
matches = matches + 1
```

After the search finishes, use

```
if matches == 0:  
    print("No matching patients "  
          "were found.")  
else:  
    print("Matches:", matches)
```

Test the program with one search that produces multiple matches and another search that produces no matches.

Key Chapter 8 ideas: string equality with `==`, substring membership with `in`, and string methods such as `.lower()`.